

Conceptos básicos de concurrencia

La clase de fundamentos: qué es la concurrencia, por qué existe, cómo se comunican y sincronizan los procesos, qué pasa a nivel de hardware, cómo se escribe en pseudocódigo y qué propiedades debe cumplir un programa concurrente.

TEMAS DEL CURSO

Conceptos básicos

Memoria compartida

Pasaje de mensajes

Lenguajes concurrentes

Programación paralela

OBJETIVOS GENERALES

- Fundamentos, sintaxis y semántica de la concurrencia.
- Integrar hardware, sistema operativo y algoritmos.
- Comunicación y sincronización: memoria compartida y mensajes.
- Vincular con procesamiento distribuido y paralelo.

PREGUNTAS QUE GUÍAN EL CURSO

- ¿Por qué es importante la concurrencia?
- ¿Cuáles son los problemas de concurrencia en los sistemas?
- ¿Cómo se resuelven usualmente esos problemas?
- ¿Cómo se resuelven a distintos niveles — hardware, SO, lenguajes, aplicaciones?
- ¿Cuáles son las herramientas disponibles?

● Introducción

● Conceptos básicos

● Hardware

● Clases de instrucciones

● Acciones atómicas

● Propiedades y fairness

MÓDULO 1

01

Introducción



Concurrencia es la capacidad de ejecutar múltiples actividades en paralelo o simultáneamente, permitiendo a distintos objetos actuar al mismo tiempo. Es un factor relevante para el diseño de hardware, sistemas operativos, multiprocesadores, computación distribuida, programación y diseño en general.

| ¿Dónde está la concurrencia?

Está en todos lados, aunque no siempre se note: un navegador que carga una página mientras atiende al usuario; varios navegadores accediendo a la misma página; el acceso a disco mientras otras aplicaciones siguen funcionando; imprimir un documento mientras se lo consulta; el teléfono avisando una llamada mientras se habla; muchos usuarios conectados al mismo sistema de reservas; cualquier objeto "inteligente" — juegos, automóviles. Los sistemas biológicos, de hecho, suelen ser masivamente concurrentes: un gran número de células evolucionando y actuando simultáneamente e independientemente — los sistemas puramente secuenciales son raros en la naturaleza. A veces se piensa en términos secuenciales solo para simplificar el diseño, pero eso va en contra de la necesidad de sistemas cada vez más poderosos y flexibles.

| Concurrencia "natural" – un ejemplo

Problema: desplegar un cartel. La forma de resolverlo cambia mucho según se piense de forma secuencial o concurrente.

1

Un solo cartel ROJO cada 3 segundos

secuencial

```
Programa Cartel
  Mientras (true)
    Demorar (3 seg)
    Desplegar cartel
  Fin mientras
Fin programa
```

2

Cartel ROJO cada 3s y AZUL cada 5s

secuencial – se complica

Un único flujo tiene que llevar la cuenta de ambos temporizadores y decidir cuál toca desplegar a continuación.

```
Programa Carteles
  Proximo_Rojo = 3
  Proximo_Azul = 5
  Actual = 0
  Mientras (true)
```

```

Si (Proximo_Rojo < Proximo_Azul)
  Demorar (Proximo_Rojo - Actual)
  Desplegar cartel ROJO
  Actual = Proximo_Rojo
  Proximo_Rojo = Proximo_Rojo + 3
sino
  Demorar (Proximo_Azul - Actual)
  Desplegar cartel AZUL
  Actual = Proximo_Azul
  Proximo_Azul = Proximo_Azul + 5
Fin mientras
Fin programa

```

⚠ Obliga a establecer un orden explícito de despliegue; el código se vuelve más complejo de desarrollar y mantener. ¿Qué pasaría con tres o más carteles?

3

Cada cartel, un proceso independiente

concurrente – más natural

En vez de un único programa que arbitra todos los temporizadores, cada cartel es un elemento independiente que actúa concurrentemente con los demás — se ejecutan dos o más algoritmos simples, en paralelo conceptual.

```

Programa Cartel (color, tiempo)
  Mientras (true)
    Demorar (tiempo segundos)
    Desplegar cartel (color)
  Fin mientras
Fin programa

```

No hay un orden preestablecido en la ejecución → **no determinismo**: ejecuciones con la misma entrada pueden generar salidas (órdenes de despliegue) distintas.

| ¿Por qué es necesaria la Programación Concurrente?

Límite físico

- No hay más ciclos de reloj para ganar → estrategia: multicore.

Estructura más natural

- El mundo no es secuencial.

- Más apropiado para actividades independientes.
- Reacción a entradas asincrónicas (ej. sensores en un STR).

Mejor respuesta

- No bloquear toda la aplicación por E/S.
- Mejor uso del hardware vía ejecución paralela.

Además habilita **sistemas distribuidos**: una misma aplicación repartida en varias máquinas, con esquemas cliente/servidor o P2P.

Objetivo de fondo: incrementar la performance mejorando los tiempos de respuesta, mediante un enfoque distinto de la arquitectura física y lógica. Ventajas: mayor velocidad de ejecución, mejor utilización de la CPU de cada procesador, y explotar la concurrencia inherente a la mayoría de los problemas reales — ajustando el modelo de hardware/software al problema real que se quiere resolver.

| Comportamientos posibles de los procesos

Un programa secuencial tiene un único flujo de control; con múltiples hilos aparece el programa concurrente o paralelo. Entre esos procesos, tres relaciones posibles: **independientes** (no se relacionan — relativamente raro y poco interesante), **competidores** (se disputan un recurso compartido — típico en SO y redes; mal resuelto, deriva en deadlock o inanición) y **cooperantes** (se combinan para resolver una tarea común, mediante sincronización).

| Secuencial, concurrente y paralelo

– fabricar un objeto de N partes

Un mismo problema — ensamblar un objeto compuesto por N módulos — resuelto de tres formas distintas.

Secuencial (1 máquina)

Fuerza un orden temporal estricto: el ensamblado final requiere N pasos de procesamiento, uno por parte.

Paralelo (N máquinas)

Si no hay dependencias, cada máquina fabrica una parte al mismo tiempo.

- + Menor tiempo total
- + Menor esfuerzo individual
- + Paralelismo real de hardware
- – Distribuir la carga (tamaños/velocidades distintas)
- – Compartir recursos sin conflictos
- – Esperarse en puntos clave

- – Comunicarse y tratar fallas
- – Elegir quién ensambla

Concurrente sin paralelismo HW

1 máquina reparte su tiempo entre los componentes → menor speedup que el caso paralelo.

- – Distribuir la carga de trabajo
- – Compartir recursos
- – Esperarse en puntos clave
- – Comunicarse
- – Recuperar el "estado" de cada proceso al retomarlo

Definición formal: **CONCURRENCIA** = concepto de software no restringido a una arquitectura particular de hardware ni a un número determinado de procesadores.

| Multiprogramación en un procesador

El caso "concurrente sin paralelismo" es, en la práctica, multiprogramación: el tiempo de CPU se reparte entre procesos (por ejemplo por time slicing). El sistema operativo controla y planifica: si el slice expira o el proceso se bloquea, ocurre un *process (context) switch*.

- ✓ **Process switch:** guardar el estado actual en memoria, encolar el proceso en ready o wait; sacar un proceso de la cabeza de la cola ready, restaurar su estado y ponerlo a correr.
- ✓ **Reanudar un proceso bloqueado:** moverlo de la cola de wait a la cola de ready.

| Programa concurrente – definición formal

Un programa concurrente especifica dos o más "programas secuenciales" que pueden ejecutarse concurrentemente en el tiempo, como tareas o procesos. Un proceso o tarea es un elemento concurrente abstracto que puede ejecutarse simultáneamente con otros si el hardware lo permite (por ejemplo, los TASKs de Ada). Un programa concurrente puede tener N procesos habilitados, y un sistema concurrente puede disponer de M procesadores, cada uno ejecutando uno o más procesos. Dos características importantes se derivan de esto: **interacción** y **no determinismo** — lo que dificulta la interpretación y el debug.

| Procesos e hilos

Proceso

Tiene su propio espacio de direcciones y sus propios recursos.

Hilo (thread / proceso liviano)

Tiene su propio contador de programa y su propia pila de ejecución, pero no controla el "contexto pesado" (por ejemplo, las tablas de página). Todos los hilos de un proceso comparten el mismo espacio de direcciones y recursos. El programador o el lenguaje deben proveer mecanismos para evitar interferencias. La concurrencia puede estar provista por el lenguaje (Java) o por el sistema operativo (C/POSIX).

02

MÓDULO 2

Conceptos básicos de concurrencia

Para que procesos concurrentes trabajen juntos hace falta que se comuniquen y se sincronicen — y ahí aparecen los problemas centrales de toda la materia: interferencia, deadlock, y las propiedades que un buen diseño debe garantizar.

| Comunicación entre procesos

Indica el modo en que se organizan y transmiten datos entre tareas concurrentes; requiere protocolos que controlen el progreso y la corrección. Los procesos se comunican de dos formas:

Memoria compartida

Los procesos intercambian información sobre memoria compartida, o actúan coordinadamente sobre datos que residen en ella. Lógicamente no pueden operar de forma simultánea, lo que obliga a bloquear y liberar el acceso. La solución más elemental es una variable de control tipo "semáforo".

Pasaje de mensajes

Es necesario establecer un canal (lógico o físico) para transmitir información, y el lenguaje debe proveer un protocolo adecuado. Para que la comunicación sea efectiva, los procesos deben "saber" cuándo tienen mensajes para leer y cuándo deben transmitir.

| Sincronización entre procesos

Es la posesión de información acerca de otro proceso para coordinar actividades. Los procesos se sincronizan de dos formas:

- ✓ **Exclusión mutua:** asegura que solo un proceso tenga acceso a un recurso compartido en un instante dado; evita que dos o más procesos estén en su sección crítica al mismo tiempo.
- ✓ **Por condición:** bloquea la ejecución de un proceso hasta que se cumple una condición dada.

El ejemplo clásico que usa ambos mecanismos a la vez: un buffer limitado con productores y consumidores.

| Interferencia

Un proceso toma una acción que invalida las suposiciones hechas por otro proceso.

EJEMPLO 1 – DIVISIÓN POR CERO

```
int x, y, z;  
process A1 { ... y = 0; ... }  
process A2 { ... if (y <> 0) z =  
x/y; ... }
```

Nunca se debería dividir por 0 — pero si A2 evalúa $y \neq 0$ justo antes de que A1 ponga $y=0$, la suposición de A2 queda invalidada.

EJEMPLO 2 – CONTADOR COMPARTIDO

```
int Público = 0  
process B1 { int E1=0;  
  for i=1..100 { esperar  
llegada;  
    E1=E1+1;  
    Público=Público+1; } }  
process B2 { int E2=0;  
  for i=1..100 { esperar  
llegada;  
    E2=E2+1;  
    Público=Público+1; } }
```

Siempre debería terminar $\text{Público} = E1 + E2$ — pero sin sincronización, dos incrementos simultáneos sobre Público pueden pisarse (se pierde una actualización).

| Prioridad y granularidad

La granularidad de una aplicación está dada por la relación entre cómputo y comunicación (grano fino vs. grano grueso, según su adaptación a la arquitectura — ver recapitulación de la conversación). Además, un proceso con mayor prioridad puede causar la suspensión (*preemption*) de otro proceso concurrente, y análogamente puede tomar un recurso compartido, obligando a retirarse al proceso que lo tenía en ese instante.

| Manejo de los recursos

Uno de los temas centrales de la programación concurrente es administrar los recursos compartidos: asignación, métodos de acceso, bloqueo y liberación,

seguridad y consistencia. Una propiedad deseable es el **fairness** — equilibrio en el acceso a recursos por todos los procesos. Dos situaciones no deseadas:

- ✓ **Inanición:** un proceso no logra acceder nunca a los recursos compartidos.
- ✓ **Overloading:** la carga asignada a un proceso excede su capacidad de procesamiento.

Y, por supuesto, evitar el deadlock.

| Problema de deadlock

Dos (o más) procesos entran en deadlock si, por error de programación, ambos quedan esperando que el otro libere un recurso compartido. La ausencia de deadlock es una propiedad necesaria en los procesos concurrentes. Existen **4 propiedades necesarias y suficientes** para que exista deadlock:

- ✓ **Recursos reusables serialmente:** los procesos comparten recursos que se usan con exclusión mutua.
- ✓ **Adquisición incremental:** los procesos retienen los recursos que ya tienen mientras esperan adquirir recursos adicionales.
- ✓ **No-preemption:** un recurso adquirido no puede quitarse por la fuerza; solo se libera voluntariamente.
- ✓ **Espera cíclica:** existe una cadena circular de procesos donde cada uno tiene un recurso que el siguiente en el ciclo está esperando.

| Requerimientos para un lenguaje concurrente

Independientemente del mecanismo de comunicación/sincronización elegido, un lenguaje de programación concurrente debe proveer primitivas para: indicar qué tareas o procesos pueden ejecutarse concurrentemente, mecanismos de sincronización, y mecanismos de comunicación entre procesos.

| Problemas asociados con la Programación Concurrente

- ✓ **Complejidad:** los procesos no son independientes y comparten recursos; usar exclusión mutua y sincronización agrega complejidad.
- ✓ **Pérdida de liveness:** los procesos pueden no estar "vivos" — síntoma de deadlock o de una mala distribución de recursos.
- ✓ **No determinismo:** el interleaving implícito hace que dos ejecuciones del mismo programa no sean necesariamente idénticas → dificultad para interpretar y debuggear.
- ✓ **Overhead:** posible reducción de performance por context switch, comunicación, sincronización.
- ✓ **Costo de desarrollo:** mayor tiempo de desarrollo y puesta a punto; es difícil paralelizar algoritmos secuenciales.
- ✓ **Adaptación al hardware:** hace falta adaptar el software concurrente al hardware paralelo para una mejora real de rendimiento.

| Concurrencia y Paralelismo

Concurrencia

Concepto de *software*, no restringido a una arquitectura de hardware particular ni a un número determinado de procesadores. Especificarla implica especificar los procesos concurrentes, su comunicación y su sincronización.

Paralelismo

Se asocia con la ejecución concurrente en *múltiples procesadores*, con el objetivo principal de reducir el tiempo de ejecución.

03

MÓDULO 3

Concurrencia a nivel de hardware

Todo lo anterior es diseño de software. Esta parte muestra su correlato físico: por qué el hardware cambió de forma, y las dos grandes familias de arquitectura sobre las que se apoya el resto de la materia.

| Límite físico en la velocidad de los procesadores

Las máquinas monoprocesador ya no pueden mejorar por la vía de acelerar un único núcleo. La respuesta de la industria: más procesadores por chip (multicore), y luego clusters de multicores — a costa de mayor consumo energético. Todo ese hardware solo se aprovecha con un uso eficiente: programación concurrente y paralela.

| Niveles de memoria

Jerarquía CPU → Caché Nivel 1 → Caché Nivel 2 → Memoria Principal: a medida que se sube, el tamaño crece pero el tiempo de acceso también. Existe gracias a la localidad temporal y espacial de los datos, y trae consigo la pregunta de la **consistencia**: si varios núcleos tienen copias en su caché de los mismos datos, hay que garantizar que se mantengan sincronizadas.

| Multiprocesadores de memoria compartida

Los procesadores interactúan modificando datos en una memoria compartida.

UMA (uniform memory access)

Bus o crossbar switch;
multiprocesadores simétricos (SMP).
Problemas de sincronización y consistencia.

NUMA (non-uniform)

Para un mayor número de procesadores distribuidos. También tiene problema de consistencia.

Ejemplos concretos: un multicore de 8 núcleos, donde cada par de núcleos comparte una caché L2, varias L2 comparten una L3, y todo converge en la memoria compartida; y una GPU, otro caso típico de multiprocesador de memoria compartida.

| Multiprocesadores con memoria distribuida

Procesadores conectados por una red, cada uno con memoria local (sin problemas de consistencia); la única interacción posible es por pasaje de mensajes. Se distinguen por su grado de acoplamiento:

- ✓ **Multicomputadores** (fuertemente acoplados): procesadores y red físicamente cerca, pocas aplicaciones a la vez, alto ancho de banda y velocidad.
- ✓ **Memoria compartida distribuida.**
- ✓ **Clusters.**
- ✓ **Redes** (débilmente acoplado).

| Un poco de historia

La concurrencia evolucionó de enfoques ad-hoc a técnicas generales, siguiendo los cambios tecnológicos.

60's

Evolución de los SO; controladores de dispositivos (canales) independientes → interrupciones, no determinismo, multiprogramación, y aparece el problema de la sección crítica.

70's

Formalización de la concurrencia en los lenguajes de programación.

80's

Redes y procesamiento distribuido.

90's

MPP, Internet, cliente/servidor, web computing.

2000's

Sistemas de tiempo real, computación móvil, cluster y multicluster computing, sistemas colaborativos, computación pervasiva y ubicua, grid computing, virtualización.

Hoy

Big data, IA, computación elástica, cloud computing, green computing, bioinformática, redes de sensores, IoT, banca electrónica...

El pseudocódigo que se usa durante toda la cursada. Un programa concurrente está formado por un conjunto de programas secuenciales; la parte secuencial se expresa con tres clases básicas — asignación, alternativa, iteración — y hace falta una clase más para representar la concurrencia misma.

Declaración de variables

```
int x = 8; int z, y;  
int a[10]; int c[3:10];  
int b[10] = ([10] 2);  
int aa[5,5]; int cc[3:10,2:9];  
int bb[5,5] = ([5] ([5] 2));
```

Asignación

```
x = e;                // simple  
x = x+1; z = x+y;     // compuesta  
a[3] = 6; aa[2,5] = a[4]; // indexada  
x = f(y) + g(6) - 7;  // llamada  
v1 := v2;             // swap  
skip;                 // no-op
```

| Alternativa

SIMPLE Y MÚLTIPLE

```
if B → S                // B "guarda" a S  
  
if B1 → S1  
[] B2 → S2  
...  
[] Bn → Sn  
fi
```

Las guardas se evalúan en orden arbitrario; elección no determinística si más de una es verdadera. Si ninguna guarda es verdadera, el if no tiene efecto.

NOTACIÓN ALTERNATIVA

```
if (cond) S;  
if (cond) S1 else S2;
```

EJERCICIO 1

```
if p>2 → p=p*2  
[] p<2 → p=p*3  
[] p==2 → p=5  
fi
```

¿Puede terminar sin efecto? ¿Qué pasa si p=2?

EJERCICIO 2

```
if p>2 → p=p*2  
[] p<2 → p=p*3  
fi
```

¿Qué pasa si p=2?

EJERCICIO 3

```
if p>2 → p=p*2  
[] p<6 → p=p+4  
[] p==4 → p=p/2  
fi
```

¿Con p = 1, 2, 3, 4, 5, 6, 7?

| Iteración

ALTERNATIVA ITERATIVA MÚLTIPLE

```
do B1 → S1  
[] B2 → S2  
...  
[] Bn → Sn  
od
```

Se evalúan y ejecutan las sentencias guardadas hasta que todas las guardas son falsas. Elección no determinística si hay más de una guarda verdadera.

FOR-ALL

```

fa cuantificadores → Instrucciones af
// cuantificador ::= var := ini to fin st B

for [i=1 to n, j=1 to n st (j mod 2=0)] S;
while (cond) S;

```

El cuerpo se ejecuta una vez por cada combinación de valores de las variables de iteración que cumple la cláusula st (such-that), si existe.

EJERCICIO – D0/0D

```

do p>0 → p=p-2
[] p<0 → p=p+3
[] p==0 → p=random(x)
od

```

¿Cuándo termina?

EJERCICIO – D0/0D

```

do p>0 → p=p-2
[] p>3 → p=p+3
[] p>6 → p=p/2
od

```

¿Con p = 0, 3, 6, 9?

FOR-ALL – EJEMPLOS

```

fa i:=1 to n, j:=i+1 to n → m[i,j]:=m[j,i] af // trasponer
fa i:=1 to n, j:=i+1 to n st a[i]>a[j] → a[i]:=a[j] af // ordenar
fa i:=1 to n → a[i]=0 af // inicializar

```

| Concurrencia – co y process

SENTENCIA CO

```

co S1 // S2 // ... // Sn oc
// ejecuta las Si concurrentemente;
// termina cuando todas terminaron

co [i=1 to n] { a[i]=0; b[i]=0 } oc // n tareas

```

SENTENCIA PROCESS

```
process A { sentencias }           // 1 proceso
process B [i=1 to n] { sentencias } // n procesos
```

Diferencia clave: process se ejecuta en background; el código que contiene un `co` espera a que termine antes de seguir con la siguiente sentencia.

¿QUÉ IMPRIME CADA CASO? ¿SON EQUIVALENTES?

```
process imprime10 {
  for [i=1 to 10] write(i);
}
```

```
process imprime1 [i=1..10] {
  write(i);
}
```

No: `imprime10` es un único proceso que imprime 1..10 en orden; `imprime1` crea 10 procesos independientes, cada uno imprime un número — el orden de salida es no determinístico.

05

MÓDULO 5

Acciones atómicas y sincronización

El núcleo teórico más denso de la clase: qué significa realmente que algo sea "atómico", por qué la intuición falla al razonar sobre programas concurrentes, y cómo se especifica formalmente la sincronización.

I Estado, historias e interleaving

Cada proceso ejecuta sentencias implementadas por una o más acciones atómicas: una transformación de estado indivisible, cuyos estados intermedios son invisibles para los demás procesos. La ejecución de un programa concurrente es un intercalado (*interleaving*) de las acciones atómicas de los procesos individuales. Una **historia** (trace) es la ejecución con un interleaving particular — el número de historias posibles es enorme, pero no todas son válidas: la interacción entre procesos determina cuáles lo son.

```
int buffer;
process 1 { int x; while(true) {
  p1.1: read(x);
  p1.2: buffer = x; } }
process 2 { int y; while(true) {
  p2.1: y = buffer;
  p2.2: print(y); } }
```

Posibles historias: $p_{11}, p_{12}, p_{21}, p_{22}, p_{11}, p_{12}, \dots$ / $p_{11}, p_{12}, p_{21}, p_{11}, p_{22}, p_{12}, \dots$ / $p_{11}, p_{21}, p_{12}, p_{22}, \dots$ / $p_{21}, p_{11}, p_{12}, \dots$ — la sincronización por condición restringe qué historias son válidas, para asegurar el orden temporal necesario.

| Atomicidad de grano fino

¿Es atómica la asignación $A=B$? **No** — se descompone en `Load PosMemB, reg / Store reg, PosMemA`. Y $X=X+X$ se descompone en `Load / Add / Store`. Una acción atómica de grano fino se implementa en hardware.

EJEMPLO 1 — 3 PROCESADORES: $X=0; Y=4; Z=2$; `C0 X=Y+Z(1) // Y=3(2) // Z=4(3) 0C`

$y=3$ y $z=4$ en todos los casos. El valor final de x depende del orden de interleaving — incluso descomponiendo (1) en sus pasos de grano fino (1.1 Load y , 1.2 Add z , 1.3 Store x):

ORDEN DE EJECUCIÓN	X RESULTANTE
(1)(2)(3) o (1)(3)(2)	6
(2)(1)(3)	5
(3)(1)(2)	8
(2)(3)(1) o (3)(2)(1)	7
(1.1)(2)(1.2)(1.3)(3)	6
(1.1)(3)(1.2)(1.3)(2)	8

...y otras combinaciones más, intercalando los pasos de grano fino de (1) con (2) y (3).

EJEMPLO 2 — 2 PROCESADORES: $X=2; Y=2$; `C0 Z=X+Y(1) // X=3; Y=4(2) 0C`

$x=3$, $y=4$ siempre. z puede ser 4, 5, 6 o 7 — pero el programa *nunca podría* parar y observar un estado donde $x+y=6$, a pesar de que $z=x+y$ sí puede terminar valiendo 6 (la lectura de x e y por parte de (1) puede ocurrir en instantes distintos, mezclando valores viejos y nuevos, sin que eso corresponda a ningún estado consistente real).

EJEMPLO 3 — "INTERLEAVING EXTREMO" (BEN-ARI & BURNS)

```
int X = 0;
process P1 { int i; for [i=1 to N] → X=X+1 }
process P2 { int i; fa [i=1 to N] → X=X+1 }
```

Dos procesos hacen, cada uno, N iteraciones de $X=X+1$. ¿Cuál puede ser el valor final de X ?

Vale 2N

Sin solapamiento — cada iteración completa (load, incrementa, store) de un proceso antes que la del otro.

1. P1: Load X

Vale N

Ambos leen antes de que ninguno escriba, en cada iteración — se pierde una actualización por iteración.

1. P1: Load X

Vale 2 (extremo)

Un proceso avanza casi todas sus iteraciones con datos viejos.

1. P1: Load X

2. P1: incrementa
3. P1: Store X
4. P2: Load X
5. P2: incrementa
6. P2: Store X

2. P2: Load X
3. P1: incrementa
4. P2: incrementa
5. P1: Store X
6. P2: Store X

2. P2: N-1 iteraciones
3. P1: incrementa
4. P1: Store X
5. P2: Load X
6. P1: resto de iteraciones
7. P2: incrementa
8. P2: Store X

Conclusión de la cátedra: **no podemos confiar en la intuición para analizar un programa concurrente.**

En la mayoría de los sistemas el tiempo absoluto no importa — con frecuencia se los actualiza con componentes más rápidos, y la corrección no debe depender de tiempos absolutos. Solo importan las secuencias: puede haber distintos interleavings de las instrucciones de los procesos, y el programa debe ser correcto para todos ellos.

SUPUESTOS DE MÁQUINA ASUMIDOS DE ACÁ EN ADELANTE

- ✓ Los tipos básicos se leen y escriben en memoria como acciones atómicas.
- ✓ Los valores se cargan en registros, se opera sobre ellos, y el resultado se guarda en memoria.
- ✓ Cada proceso tiene su propio conjunto de registros (de ahí el context switching).
- ✓ Los resultados intermedios de una expresión compleja se guardan en registros o en memoria privada del proceso.

Referencia crítica: una expresión que no referencia ninguna variable alterada por otro proceso se evalúa atómicamente (aunque requiera varias acciones de grano fino). Como los programas concurrentes normalmente no son disjuntos, hace falta un requisito más débil: una referencia crítica es una referencia a una variable modificada por otro proceso; se asume que toda referencia crítica es a una variable simple, leída y escrita atómicamente.

| Propiedad "A lo sumo una vez" (ASV)

Puede haber a lo sumo una variable compartida referenciada, y a lo sumo una vez. Formalmente, una asignación $x = e$ satisface ASV si (1) e contiene a lo sumo una referencia crítica y x no es referenciada por otro proceso, o (2) e no contiene referencias críticas (en cuyo caso x sí puede ser leída por otro proceso). Una expresión que no está en una asignación satisface ASV si no contiene más de una referencia crítica. Si una asignación cumple ASV, su ejecución *parece* atómica: la variable compartida se lee o escribe una única vez.

```
int x=0, y=0;
co x=x+1 // y=y+1 oc;
```

Sin referencias críticas en ningún proceso.

Garantizado: **x=1, y=1** en todas las historias.

```
int x=0, y=0;
co x=y+1 // y=y+1 oc;
```

1er proceso: 1 ref. crítica (y). 2do: ninguna.

Garantizado: **y=1** siempre; **x=1 o 2**.

```
int x=0, y=0;
co x=y+1 // y=x+1 oc;
```

Ninguna asignación satisface ASV.

Posibles: **x=1,y=2** o **x=2,y=1**. Nunca **x=1,y=1** — sería interferencia.

| Especificación de la sincronización

Cuando una expresión o asignación no satisface ASV, con frecuencia hace falta ejecutarla atómicamente — en general, ejecutar una secuencia de sentencias como una única acción atómica (sincronización por exclusión mutua). Notación: **<e>** indica que **e** se evalúa atómicamente; **await (B) S**; especifica sincronización, donde **B** es la condición de demora y **S** una secuencia que se garantiza que termina — se asegura que **B** es true cuando **S** comienza, y ningún estado interno de **S** es visible para otros procesos.

INCONDICIONAL Y CONDICIONAL

```
<x=x+1>; <y=y+1>;          // exclusión mutua
await (s>0) s=s-1;         // await general
await (count>0)            // sync por condición
```

Si **B** satisface ASV, puede implementarse como busy waiting: **do (not B) → skip od** (equiv. **while (not B);**).

PRODUCTOR/CONSUMIDOR — BUFFER DE TAMAÑO N

```
cant: int = 0; Buffer: cola;
process Productor { while(true) {
  Generar Elemento;
```

```
<await(cant<N); push(buffer,elem); cant++>
} }
process Consumidor { while(true) {
  <await(cant>0); pop(buffer,elem); cant-->
  Consumir Elemento;
} }
```

MÓDULO 6

06

Propiedades y fairness

Cierra la teoría con el vocabulario formal para evaluar si un programa concurrente está bien diseñado, y con la pieza que hace que las demostraciones de "eventual entrada" tengan sentido: el fairness del scheduler.

| Propiedades de seguridad y vida

Una propiedad de un programa concurrente es un atributo verdadero en cualquiera de sus historias de ejecución. Toda propiedad se formula en dos clases:

Seguridad (safety)

Nada malo le ocurre a un proceso: asegura estados consistentes. Una falla de seguridad indica que algo anda mal. Ejemplos: exclusión mutua, ausencia de interferencia entre procesos, partial correctness.

Vida (liveness)

Eventualmente ocurre algo bueno: la actividad progresa, no hay deadlocks. Una falla de vida indica que las cosas dejan de ejecutar. Ejemplos: terminación, que un pedido de servicio sea atendido, que un mensaje llegue a destino, que un proceso alcance su SC — dependen de las políticas de scheduling.

Pregunta abierta que deja la cátedra: ¿qué pasa con la *total correctness*? (intuitivamente, seguridad + vida juntas).

| Fairness y políticas de scheduling

Fairness busca garantizar que los procesos tengan chance de avanzar, sin importar lo que hagan los demás. Una acción atómica es *elegible* si es la próxima acción del proceso que sería ejecutada; con varios procesos, hay varias acciones elegibles a la vez. Una política de scheduling determina cuál se ejecuta.

```
bool continue = true;
co while (continue); // continue = false; oc
```

Con una política que asigna el procesador a un proceso hasta que termina o se demora, el primer proceso (el while infinito) nunca cede el procesador, y el segundo nunca llega a poner `continue=false` → el programa no termina bajo esa política, aunque sí terminaría con round-robin.

I Tres niveles de fairness

1

Incondicional

la más débil

Toda acción atómica *incondicional* que es elegible eventualmente se ejecuta. Round-robin es incondicionalmente fair en monoprocesador; la ejecución paralela lo es en multiprocesador.

2

Débil

(1) + condición estable

(1) Es incondicionalmente fair, y (2) toda acción atómica *condicional* que se vuelve elegible eventualmente se ejecuta — asumiendo que su condición se vuelve true y *permanece* true hasta que el proceso la ve.

⚠ No alcanza para asegurar que un await elegible siempre se ejecute si la guarda puede oscilar (`false→true→false`) mientras el proceso está demorado.

3

Fuerte

la más exigente

(1) Es incondicionalmente fair, y (2) toda acción atómica condicional elegible eventualmente se ejecuta porque su guarda se vuelve true con *infinita frecuencia*.

```
bool continue=true, try=false;
co while (continue) { try=true; try=false; }
// await (try) continue=false
oc
```

¿Termina este programa? Solo bajo un scheduling fuertemente fair. En la práctica es difícil tener una política fuertemente fair que además sea práctica: con 1 procesador, alternar las acciones de los procesos sería fuertemente fair pero impráctico; round-robin es práctico, pero no es fuertemente fair.

CIERRE

De acá en adelante

Con estos conceptos — comunicación, sincronización, granularidad, atomicidad, ASV, safety/liveness y fairness — quedan sentadas las bases para todo lo que sigue: la Teoría 2 aplica exactamente estas ideas al primer mecanismo concreto de sincronización por memoria compartida, **locks y barreras**.

Basado en `teorial.pdf` — Programación Concurrente, Facultad de Informática UNLP.